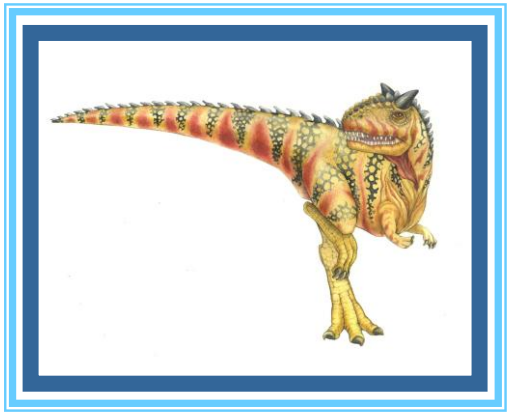
Chapter 6: Synchronization Tools

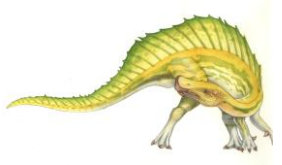


Important Note (1)

- Do notserialize your multi-process or multi-threaded code by placing **wait()** or **pthread_join()** to force an execution order.
  - **Clarify with your teacher if you unable to get this?**

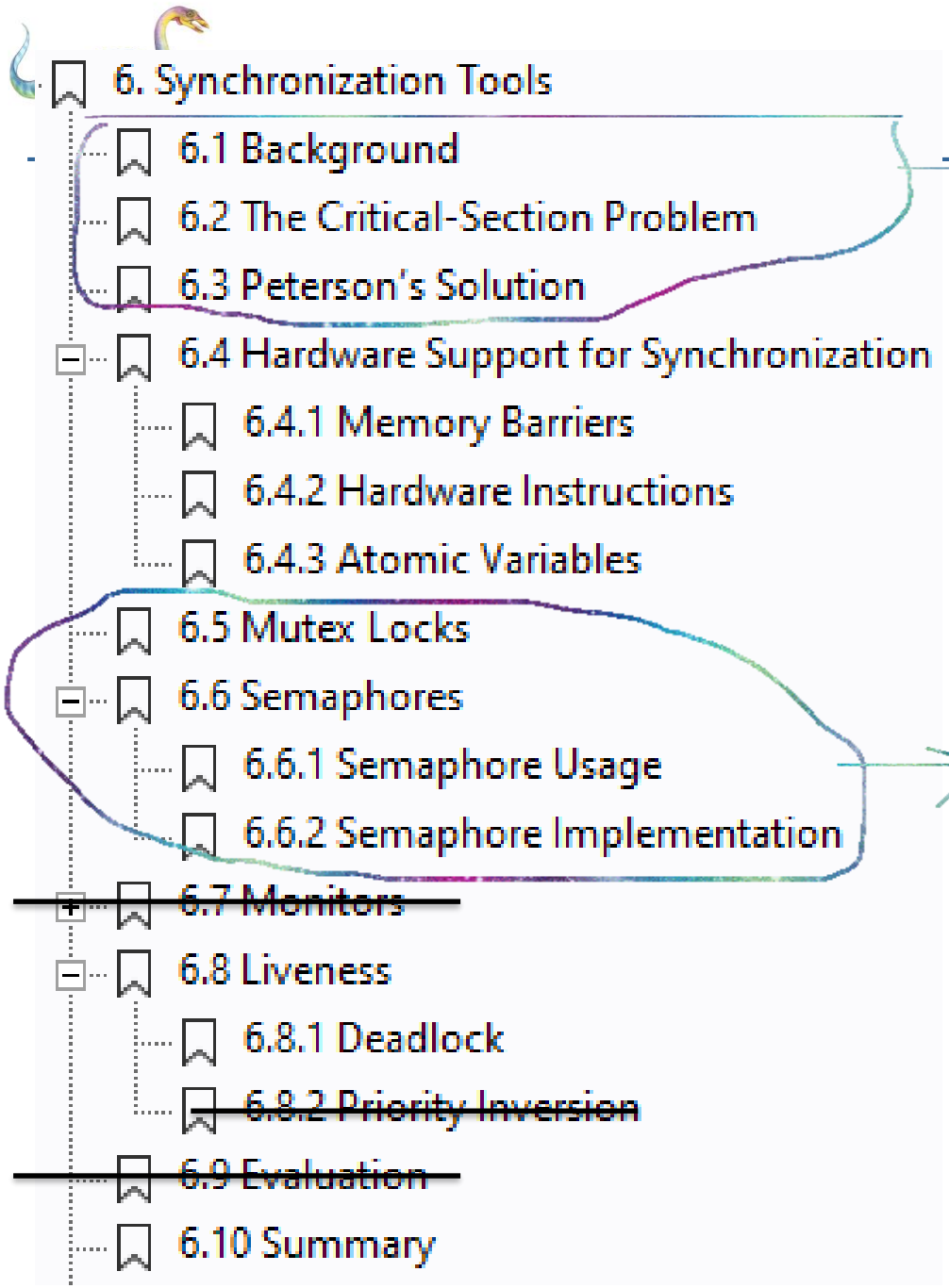
```
int main()
{
            pthread_t th1;
            pthread_t th2;
            pthread_create(&th1, NULL, function, NULL);
            pthread_create(&th2, NULL, function, NULL);
            pthread_join(th1, NULL);
            pthread_join(th2, NULL);
}
```

- The above code only ordering guarantees here are that pthread_join(th1, NULL); will not return until thread 1 has exited and pthread_join(th2, NULL); will not return until thread 2 has exited. Consequently, the main() function will not return (and the process will not exit) until both thread 1 and thread 2 have exited.



Chapter 4 Threads & Concurrency

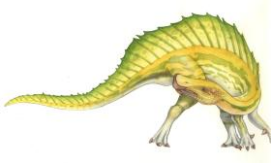
Chapter 6 Synchronization Tools



Concrete Theoretical understanding needed with textbook examples.

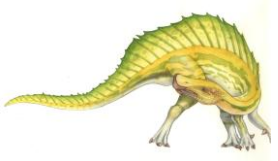
Student must have a basic understanding of these concepts

Concrete Theoretical understanding needed
+
Student should be able to write Pthread C code using Mutex Locks and Semaphores.



Race Condition

- Occurs when multiple processes/threads access shared data concurrently, and outcome depends on non-deterministic order of execution.
- Leads to inconsistent results if synchronization mechanisms. **How?**
- Critical in multi-threaded/multi-process programs.



Race Condition

Producer

```
while (true) {
     /* produce an item in next_produced */

     while (count == BUFFER_SIZE)
        ; /* do nothing */

     buffer[in] = next_produced;
     in = (in + 1) % BUFFER_SIZE;
     count++;
}
```

Consumer

```
while (true) {
     while (count == 0)
        ; /* do nothing */

     next_consumed = buffer[out];
     out = (out + 1) % BUFFER_SIZE;
     count--;

     /* consume the item in next_consumed */
}
```

1. **Producer Code:**
   - Produces item, waits if buffer full (`count == BUFFER_SIZE`).
   - Writes item to `buffer[in]` , updates `in` (circular buffer), increments `count` .
2. **Consumer Code:**
   - Waits if buffer empty (`count == 0`).
   - Reads item from `buffer[out]` , updates `out` , decrements `count` .

Race Condition:

- Occurs because `count++` (producer) and `count--` (consumer) are **non-atomic** operations.
- Both threads may read/write `count` simultaneously, leading to incorrect buffer state.
- Example: Producer increments `count` but consumer reads stale `count` , causing buffer overflow/underflow.

Why?:

- No synchronization (e.g., mutex, semaphore) ensures mutual exclusion for shared variables (`count` , `buffer` , `in` , `out`).
- Interleaved execution of producer/consumer threads corrupts shared state.

Race Condition

```
while (true) {
     /* produce an item in next_produced */

     while (count == BUFFER_SIZE)
        ; /* do nothing */

     buffer[in] = next_produced;
     in = (in + 1) % BUFFER_SIZE;
     count++;
}
```

$register_1$ = count
$register_1$ = $register_1$ + 1
count = $register_1$

```
while (true) {
     while (count == 0)
        ; /* do nothing */

     next_consumed = buffer[out];
     out = (out + 1) % BUFFER_SIZE;
     count--;

     /* consume the item in next_consumed */
}
```

$register_2$ = count
$register_2$ = $register_2$ − 1
count = $register_2$

A situation where several processes access and manipulate the same data concurrently and the outcome of the execution depends on the particular order in which the access takes place, is called a **race condition**.

Although the producer and consumer routines shown above are correct separately, they may not function correctly when executed concurrently. As an illustration, suppose that the value of the variable count is currently 5 and that the producer and consumer processes concurrently execute the statements "count++" and "count--". Following the execution of these two statements, the value of the variable count may be 4, 5, or 6! The only correct result, though, is count == 5, which is generated correctly if the producer and consumer execute separately.

Race Condition

```
while (true) {
     /* produce an item in next_produced */

     while (count == BUFFER_SIZE)
        ; /* do nothing */

     buffer[in] = next_produced;
     in = (in + 1) % BUFFER_SIZE;
     count++;
}
```

$register_1 = count$
$register_1 = register_1 + 1$
$count = register_1$

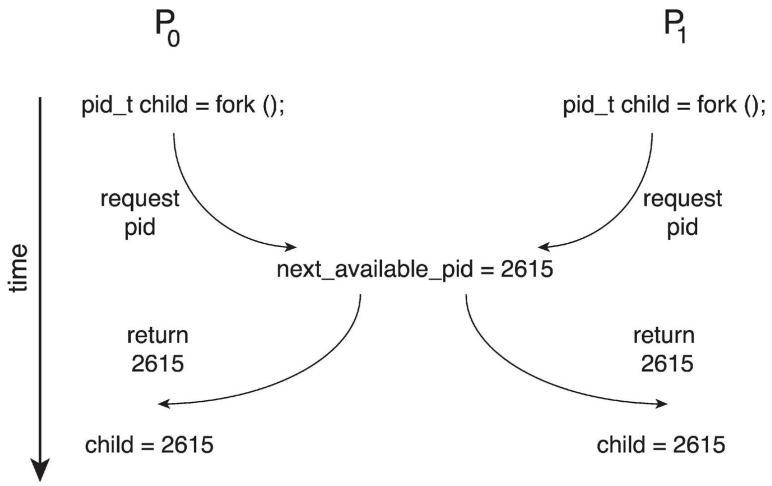
```
while (true) {
     while (count == 0)
        ; /* do nothing */

     next_consumed = buffer[out];
     out = (out + 1) % BUFFER_SIZE;
     count--;

     /* consume the item in next_consumed */
}
```

$register_2 = count$
$register_2 = register_2 - 1$
$count = register_2$

T_0:	*producer*	execute	$register_1 = count$	$\{register_1 = 5\}$
T_1:	*producer*	execute	$register_1 = register_1 + 1$	$\{register_1 = 6\}$
T_2:	*consumer*	execute	$register_2 = count$	$\{register_2 = 5\}$
T_3:	*consumer*	execute	$register_2 = register_2 - 1$	$\{register_2 = 4\}$
T_4:	*producer*	execute	$count = register_1$	$\{count = 6\}$
T_5:	*consumer*	execute	$count = register_2$	$\{count = 4\}$

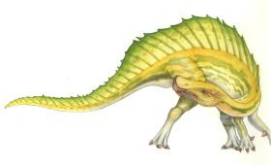
Race Condition

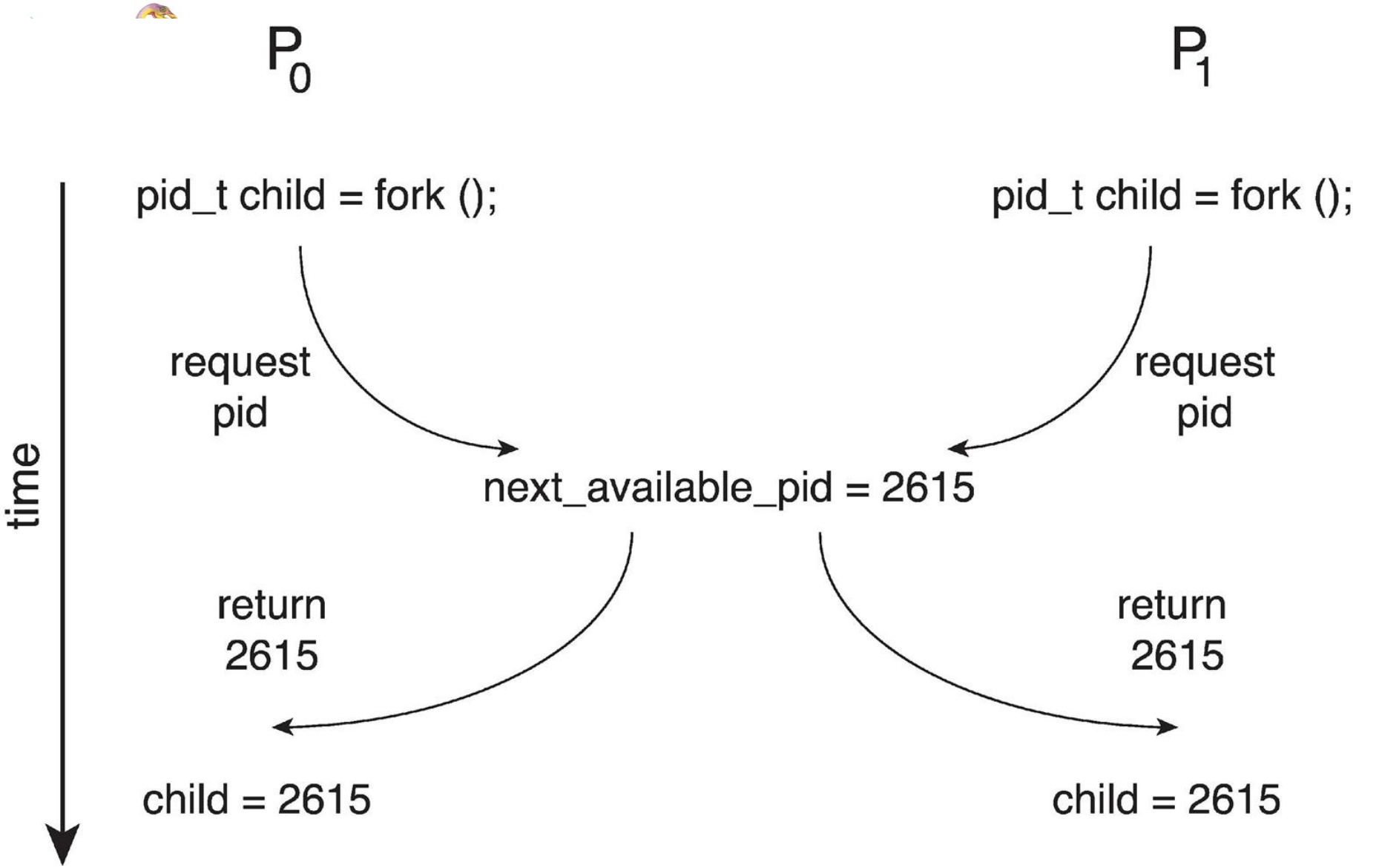
- Processes P_0 and P_1 are creating child processes using the `fork()` system call
- Race condition on kernel variable `next_available_pid` which represents the next available process identifier (pid)



See larger diagram on the next slide.

- Unless there is a mechanism to prevent P_0 and P_1 from accessing the variable `next_available_pid` the same pid could be assigned to two different processes!





Explain race condition here in your own words

Critical Section Problem

```
while (true) {
        entry section
            critical section
        exit section
            remainder section
}
```

We begin our consideration of process synchronization by discussing the so-called critical-section problem. Consider a system consisting of n processes $\{P_0, P_1, \ldots, P_{n-1}\}$. Each process has a segment of code, called a **critical section**, in which the process may be accessing — and updating — data that is shared with at least one other process. The important feature of the system is that, when one process is executing in its critical section, no other process is allowed to execute in its critical section. That is, no two processes are executing in their critical sections at the same time. The *critical-section problem* is to design a protocol that the processes can use to synchronize their activity so as to cooperatively share data. Each process must request permission to enter its critical section. The section of code implementing this request is the **entry section**. The critical section may be followed by an **exit section**. The remaining code is the **remainder section**. The general structure of a typical process is shown in Figure 6.1. The entry section and exit section are enclosed in boxes to highlight these important segments of code.

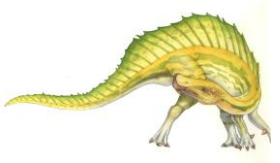
Critical Section Problem

P0/T0

```
while (true) {
        entry section
            critical section
        exit section
            remainder section
}
```

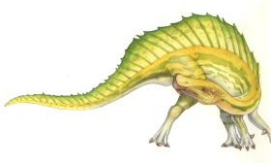
P1/T1

```
while (true) {
        entry section
            critical section
        exit section
            remainder section
}
```



How we solve CS Problem in Code?

- **Mutex**: Exclusive access to a resource.
- **Semaphore**: Manages access to a pool of resources or signals.
- **Condition Variable**: Coordinates threads based on conditions, always paired with a mutex.



Critical Section Problem

```
while (true) {
        entry section
            critical section
        exit section
            remainder section
}
```

A solution to the critical-section problem must satisfy the following three requirements:

1. **Mutual exclusion**. If process P_i is executing in its critical section, then no other processes can be executing in their critical sections.
2. **Progress**. If no process is executing in its critical section and some processes wish to enter their critical sections, then only those processes that are not executing in their remainder sections can participate in deciding which will enter its critical section next, and this selection cannot be postponed indefinitely.
3. **Bounded waiting**. There exists a bound, or limit, on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted.

What is progress?

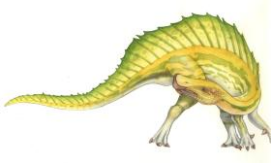
In the context of solving the critical section problem, "progress" refers to the assurance that if no process is currently executing within its critical section and some processes wish to enter their critical sections, then one of these processes must eventually be able to enter its critical section. In other words, progress ensures that the system does not deadlock or indefinitely postpone access to the critical section.

To achieve progress in mutual exclusion solutions for the critical section problem, it's important to consider the following:

1. **Fairness**: A fair solution ensures that every process that desires entry into the critical section gets a fair chance to enter, rather than certain processes consistently obtaining access while others are indefinitely blocked. Fairness helps prevent starvation, where some processes are always denied access to the critical section.
2. **No Blocking Indefinitely**: Progress guarantees that a process waiting to enter its critical section will eventually do so, even if other processes are currently in their critical sections or in their remainder sections. This means that no process should be blocked indefinitely from entering its critical section due to the actions of other processes.
3. **Avoidance of Deadlocks**: Deadlock occurs when two or more processes are blocked indefinitely, waiting for a resource held by another process that is also waiting. Progress ensures that deadlock situations are avoided by breaking circular dependencies or ensuring that processes can make progress even if they can't immediately access the critical section.

Interrupt-based Solution

- Entry section: disable interrupts
- Exit section: enable interrupts
- Will this solve the problem?
  - What if the critical section is code that runs for an hour?
  - Can some processes starve – never enter their critical section.
  - What if there are two CPUs?



Software Solution 1

- Two process solution
- Assume that the **`load`** and **`store`** machine-language instructions are atomic; that is, cannot be interrupted
- The two processes share one variable:
  - **`int turn;`**
- The variable **`turn`** indicates whose turn it is to enter the critical section
- initially, the value of **`turn`** is set to *i*

- Mutual exclusion is preserved

  $\mathbf{P_i}$ enters critical section only if:

  `turn = i`

  and **`turn`** cannot be both 0 and 1 at the same time
- What about the Progress requirement?
- What about the Bounded-waiting requirement?

```
while (true){

        while (turn = = j);

        /* critical section */

        turn = j;

        /* remainder section */

}
```

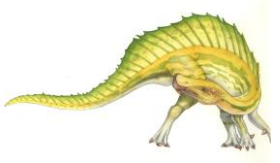
Peterson's Solution

Peterson's solution is one of the early algorithms designed to achieve mutual exclusion in a shared memory environment. Proposed by Gary L. Peterson in 1981, this solution is simple yet effective, utilizing only shared variables and atomic operations like test-and-set or compare-and-swap.

1. **Shared Variables**: Peterson's solution relies on two shared variables:
   - `` `int turn` ``: This variable indicates whose turn it is to enter the critical section. It alternates between the two processes.
   - `` `bool flag[2]` ``: An array of boolean flags, one for each process. Each flag indicates whether the corresponding process wants to enter the critical section.
2. **Algorithm Overview**:
   - Each process executes the following steps to enter the critical section:
     2.1. Sets its flag to indicate its intention to enter the critical section.
     2.2. Sets `` `turn` `` to the ID of the other process.
     2.3. Checks if the other process is currently in its critical section (`` `flag[1 - processID]` `` is true) and if it's the other process's turn (`` `turn == 1 - processID` ``). If both conditions are true, it enters a busy-wait loop, waiting for the other process to finish.
     2.4. If it can't enter the critical section immediately, it waits until either the other process has finished its critical section or it becomes its turn.
   - After finishing the critical section, the process resets its flag, allowing the other process to enter.

Peterson's Solution

- Two process solution
- Assume that the **`load`** and **`store`** machine-language instructions are atomic; that is, cannot be interrupted
- The two processes share two variables:
  - **`int turn;`**
  - **`boolean flag[2]`**
- The variable **`turn`** indicates whose turn it is to enter the critical section
- The **`flag`** array is used to indicate if a process is ready to enter the critical section.
  - **`flag[i] = true`** implies that process P_i is ready!



Algorithm for Process P_i

```
while (true) {
        flag[i] = true;
        turn = j;
        while (flag[j] && turn == j)
                ;

                /* critical section */

        flag[i] = false;

                /*remainder section */
}
```

Figure 6.3 The structure of process P_i in Peterson's solution.

Dry run of Peterson's solution

P0

Here i=0, so j should be 1.

P1

Here i=1, so j should be 0;

i) turn indicates whose turn it is to enter the critical section. ii) Flag[0]/Flag[1] indicates if P0/P1 is ready to enter the critical section.

***Turn is one variable. If both processes try to update it, the last update will decide the value that will be considered by the while loop.

```
while (true) {
    flag[i] = true;
    turn = j;
    while (flag[j] && turn == j)
        ;

        /* critical section */

    flag[i] = false;

        /*remainder section */
}
```

```
while (true) {
    flag[i] = true;
    turn = j;
    while (flag[j] && turn == j)
        ;

        /* critical section */

    flag[i] = false;

        /*remainder section */
}
```

Peterson's Solution

6.3 Peterson's Solution

Peterson's solution is restricted to two processes that alternate execution between their critical sections and remainder sections. The processes are numbered P_0 and P_1. For convenience, when presenting P_i, we use P_j to denote the other process; that is, `j` equals $1 - i$.

Peterson's solution requires the two processes to share two data items:

```
int turn;
boolean flag[2];
```

The variable `turn` indicates whose turn it is to enter its critical section. That is, if `turn == i`, then process P_i is allowed to execute in its critical section. The `flag` array is used to indicate if a process is ready to enter its critical section. For example, if `flag[i]` is `true`, P_i is ready to enter its critical section. With an explanation of these data structures complete, we are now ready to describe the algorithm shown in Figure 6.3.

To enter the critical section, process P_i first sets `flag[i]` to be `true` and then sets `turn` to the value `j`, thereby asserting that if the other process wishes to enter the critical section, it can do so. If both processes try to enter at the same time, `turn` will be set to both `i` and `j` at roughly the same time. Only one of these assignments will last; the other will occur but will be overwritten immediately. The eventual value of `turn` determines which of the two processes is allowed to enter its critical section first.

```
while (true) {
    flag[i] = true;
    turn = j;
    while (flag[j] && turn == j);

        /* critical section */

    flag[i] = false;

        /*remainder section */
}
```

Figure 6.3 The structure of process P_i in Peterson's solution.

Correctness of Peterson's Solution

We now prove that this solution is correct. We need to show that:

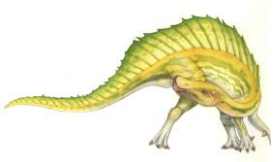
1. Mutual exclusion is preserved.
2. The progress requirement is satisfied.
3. The bounded-waiting requirement is met.

1. Mutual exclusion is preserved.

To prove property 1, we note that each P_i enters its critical section only if either `flag[j] == false` or `turn == i`. Also note that, if both processes can be executing in their critical sections at the same time, then `flag[0] == flag[1] == true`. These two observations imply that P_0 and P_1 could not have successfully executed their `while` statements at about the same time, since the value of `turn` can be either 0 or 1 but cannot be both. Hence, one of the processes—say, P_j—must have successfully executed the while statement, whereas P_i had to execute at least one additional statement ("`turn == j`"). However, at that time, `flag[j] == true` and `turn == j`, and this condition will persist as long as P_j is in its critical section; as a result, mutual exclusion is preserved.

2. The progress requirement is satisfied.
3. The bounded-waiting requirement is met.

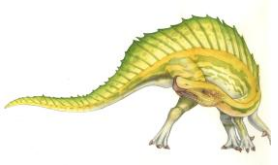
To prove properties 2 and 3, we note that a process P_i can be prevented from entering the critical section only if it is stuck in the `while` loop with the condition `flag[j] == true` and `turn == j`; this loop is the only one possible. If P_j is not ready to enter the critical section, then `flag[j] == false`, and P_i can enter its critical section. If P_j has set `flag[j]` to `true` and is also executing in its `while` statement, then either `turn == i` or `turn == j`. If `turn == i`, then P_i will enter the critical section. If `turn == j`, then P_j will enter the critical section. However, once P_j exits its critical section, it will reset `flag[j]` to `false`, allowing P_i to enter its critical section. If P_j resets `flag[j]` to `true`, it must also set `turn` to `i`. Thus, since P_i does not change the value of the variable `turn` while executing the `while` statement, P_i will enter the critical section (progress) after at most one entry by P_j (bounded waiting).



Peterson's Solution and Modern Architecture

Software-based solutions for mutual exclusion are not guaranteed to work on modern computer architectures.

- Peterson's solution is not guaranteed to work on modern computer architectures for the primary reason that, to improve system performance, processors and/or compilers may reorder read and write operations that have no dependencies.
- For a single threaded application, this reordering is immaterial as far as program correctness is concerned, as the final values are consistent with what is expected.
- But for a multithreaded application with shared data, the reordering of instructions may render inconsistent or unexpected results.



Instruction Reordering

- Done by **CPU hardware** or **compiler** for performance (out-of-order execution, pipelining).
- Reorders **instructions within a thread, as long as dependencies allow.**
- **In multithreaded apps**, this can cause **incorrect observation of shared data** across threads if synchronization is not enforced.
- **Reordering is legal from the local thread's view**, but it may **violate the global (shared) memory consistency.**

Example:

assembly

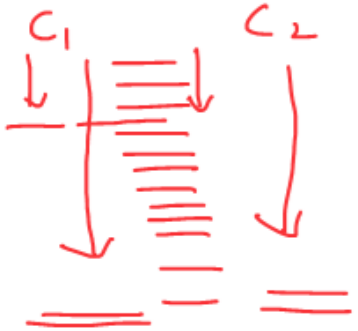
```
1. LOAD R1, [MEM1]   ; slow memory access
2. ADD  R2, R3, R4   ; independent of LOAD
```

The CPU can **execute ADD first**, even though it appears after LOAD in program order.

Instruction Reordering

Reordering of Instructions:

- Modern multicore processors **reorder instructions** (loads, stores, computations) to optimize performance.
- Done by **out-of-order execution** and **memory reordering** (e.g., store buffers, cache effects).
- Ensures **logical correctness** but may violate **program order** visible to other threads.



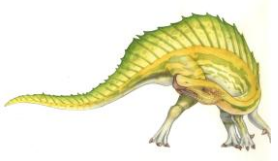
Why?:

- Improves **throughput** and **utilization** of CPU resources.
- Can cause **visibility issues** in multithreaded programs if not handled with **memory barriers** or **synchronization primitives**.

Example:

- Thread A writes `x = 1`, then `y = 2`.
- Thread B may see `y = 2` before `x = 1` due to reordering.

- Multi core CPU.
- Each core is super pipelined & superscalar.



Instruction Reordering and Peterson's Solution

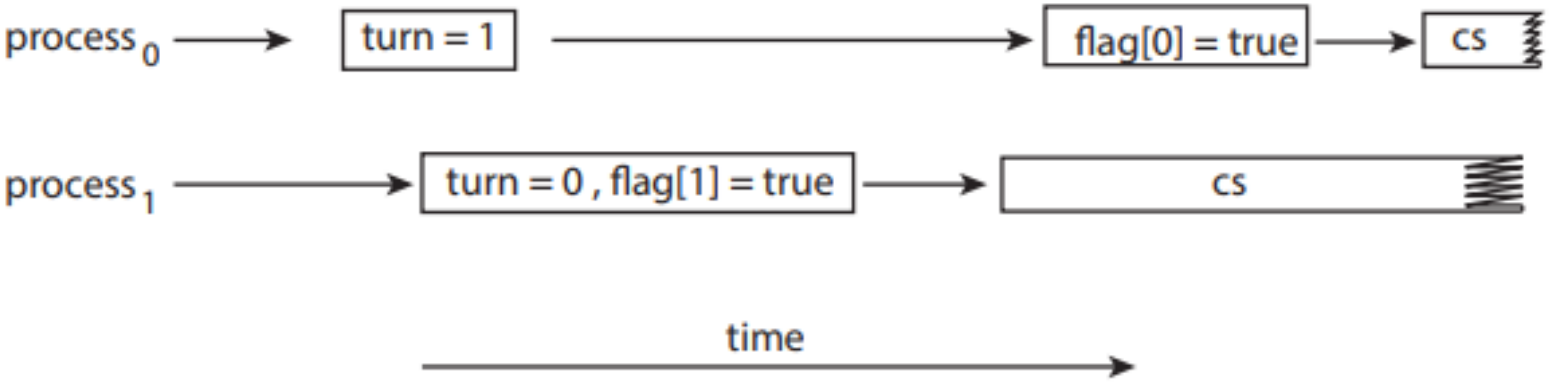


Figure 6.4 The effects of instruction reordering in Peterson's solution.

Consider what happens if the assignments of the first two statements that appear in the entry section of Peterson's solution in Figure 6.3 are reordered; it is possible that both threads may be active in their critical sections at the same time, as shown in Figure 6.4.

Figure 6.3

```
while (true) {
    flag[i] = true;
    turn = j;
    while (flag[j] && turn == j)
        ;

        /* critical section */

    flag[i] = false;

        /*remainder section */
}
```

Modern Architecture Example

As an example, consider the following data that are shared between two threads:

```
boolean flag = false;
int x = 0;
```

where Thread 1 performs the statements

```
while (!flag)
   ;
print x;
```

and Thread 2 performs

```
x = 100;
flag = true;
```

- The expected behavior is, of course, that Thread 1 outputs the value 100 for variable x.
- If due to reordering of instructions in Thread 2 so that flag is assigned true before assignment of x = 100.
  - Thread 1 would output 0 for variable x.
- The processor may also reorder the statements issued by Thread 1 and load the variable x before loading the value of flag.
  - Thread 1 would output 0 for variable x even if the instructions issued by Thread 2 were not reordered.

Modern Architecture Example (Cont.)

reorder read and write operations that have no dependencies.

As an example, consider the following data that are shared between two threads:

```
boolean flag = false;
int x = 0;
```

where Thread 1 performs the statements

```
while (!flag)
   ;
print x;
```

and Thread 2 performs

```
x = 100;
flag = true;
```

The expected behavior is, of course, that Thread 1 outputs the value 100 for variable x. However, as there are no data dependencies between the variables flag and x, it is possible that a processor may reorder the instructions for Thread 2 so that flag is assigned true before assignment of x = 100. In this situation, it is possible that Thread 1 would output 0 for variable x. Less obvious is that the processor may also reorder the statements issued by Thread 1 and load the variable x before loading the value of flag. If this were to occur, Thread 1 would output 0 for variable x even if the instructions issued by Thread 2 were not reordered.

Memory Barrier Instructions

- Explicit **instructions to stop or restrict reordering** of loads/stores.
- Used to **enforce visibility guarantees** between threads.
- Prevent **load/store reordering across the barrier**, ensuring **happens-before relationships**.

Modern CPUs reorder memory operations for performance, but this can break correctness in multithreaded programs unless synchronization primitives (locks, semaphores) use memory barriers internally.

📌 Types of Memory Barriers:

1. **Load Barrier (Read Barrier)**: Prevents loads before the barrier from being reordered after it.
2. **Store Barrier (Write Barrier)**: Prevents stores before the barrier from being reordered after it.
3. **Full Barrier (Memory Fence)**: Prevents **both** loads and stores from being reordered across it.

Concept	Scope	Effect on Multithreading
Instruction Reordering	Compiler/CPU internal	Can break data visibility without synchronization
Memory Barrier	Explicit instruction	Ensures correct order of memory operations across threads

Memory Barrier Example

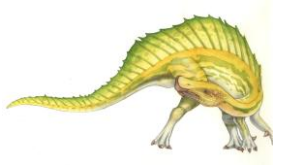
- Returning to the example of slides 6.17 - 6.18
- We could add a memory barrier to the following instructions to ensure Thread 1 outputs 100:
- Thread 1 now performs

```
while (!flag)
  memory_barrier();
print x
```

- Thread 2 now performs

```
x = 100;
memory_barrier();
flag = true
```

- For Thread 1 we are guaranteed that that the value of `flag` is loaded before the value of `x`.
- For Thread 2 we ensure that the assignment to `x` occurs before the assignment `flag`.

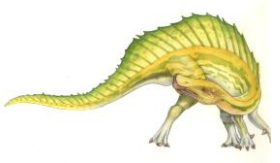


Hardware or Atomic Instructions

Hardware instructions for synchronization are necessary in multicore CPUs to ensure **atomicity**, **consistency**, and **ordering** of operations across threads. Without hardware support:

1. **Race Conditions**: Concurrent threads could corrupt shared data.
2. **Inefficiency**: Software-only synchronization (e.g., busy-waiting) wastes CPU cycles.
3. **Memory Consistency**: Cores might see inconsistent views of memory due to caching and reordering.

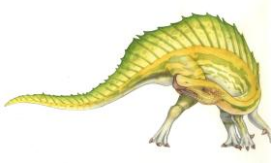
Hardware instructions (e.g., atomic operations, memory barriers) provide low-level, efficient mechanisms to coordinate threads, prevent data races, and maintain correct program behavior in parallel execution.



Hardware or Atomic Instructions

Atomic Instructions

- **Compare-and-Swap (CAS)**: This instruction atomically compares the value at a memory location with an expected value and, if they match, updates the memory location with a new value. CAS is fundamental for implementing locks, mutexes, and other synchronization primitives.
- **Load-Linked/Store-Conditional (LL/SC)**: These instructions work as a pair. `Load-Linked` loads a value from memory, and `Store-Conditional` stores a new value only if the memory location has not been modified since the `Load-Linked`. This is another way to implement atomic operations.
- **Fetch-and-Add**: This instruction atomically increments a value in memory and returns the original value. It is useful for implementing counters and other simple synchronization mechanisms.



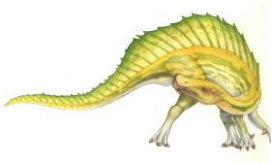
Hardware or Atomic Instructions

2. Thread-Local Storage (TLS)

- **TLS Support**: While not directly a synchronization mechanism, hardware support for thread-local storage allows each thread to have its own private data, reducing the need for synchronization in some cases.

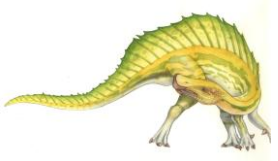
3. Synchronization Primitives in Hardware

- **Hardware Spinlocks**: Some architectures provide hardware support for spinlocks, where a core can wait for a lock to be released without consuming excessive CPU cycles.
- **Hardware Semaphores**: Similar to spinlocks, some systems provide hardware support for semaphores, which can be used for more complex synchronization patterns.



Mutex Locks

- OS designers build software tools to solve critical section problem
- Simplest is mutex lock
  - Boolean variable indicating if lock is available or not
- Protect a critical section by
  - First **`acquire()`** a lock
  - Then **`release()`** the lock
- Calls to **`acquire()`** and **`release()`** must be **atomic**
  - Usually implemented via hardware atomic instructions such as compare-and-swap.
- But this solution requires **busy waiting**
  - This lock therefore called a **spinlock**



Busy Waiting

Busy waiting is a synchronization technique where a thread repeatedly checks (spins) for a condition to become true, consuming CPU cycles while waiting. Here's a compact technical breakdown:

Why Busy Waiting?

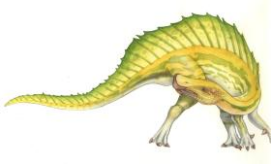
- **Low Latency**: Avoids the overhead of context switching or sleeping, making it suitable for very short waits.
- **Simplicity**: Easy to implement for simple synchronization tasks.

How It Works

1. **Spin Loop**: The thread continuously checks a condition (e.g., a flag or lock) in a loop:

```
while (!condition) {} // Busy wait
```

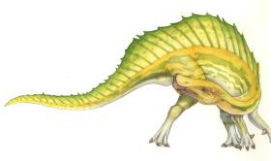
2. **No Yielding**: The thread does not yield the CPU, keeping the core busy.



Busy Waiting

Technical Details:

- CPU Usage: Consumes 100% CPU on the waiting core, wasting resources.
- No OS Involvement: Does not involve the OS scheduler, reducing latency for very short waits.
- Starvation Risk: If the condition is never met, the thread spins indefinitely.



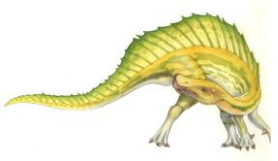
Busy Waiting

Use cases:

- Short Waits: Ideal for very short critical sections or low-contention scenarios.
- Real-Time Systems: Used in real-time systems where predictable latency is critical.
- Spinlocks: Common in kernel-level programming for lightweight synchronization.

Alternatives:

- Blocking Waits: Use OS primitives like pthread_cond_wait or semaphores to sleep the thread, freeing CPU resources.



Solution to CS Problem Using Mutex Locks

Mutex with Dynamic Initialization

```c
#include <pthread.h>

pthread_mutex_t mutex;
int shared_data = 0;

void* thread_func(void* arg) {
    pthread_mutex_lock(&mutex); // Lock
    shared_data++; // Critical section
    pthread_mutex_unlock(&mutex); // Unlock
    return NULL;
}

int main() {
    pthread_mutex_init(&mutex, NULL); // Dynamic initialization
    pthread_t thread;
    pthread_create(&thread, NULL, thread_func, NULL);
    pthread_join(thread, NULL);
    pthread_mutex_destroy(&mutex); // Cleanup
    return 0;
}
```

Solution to CS Problem Using Mutex Locks

Mutex with Trylock

```c
#include <pthread.h>

pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

void* thread_func(void* arg) {
    if (pthread_mutex_trylock(&mutex) == 0) { // Try to lock
        // Critical section
        pthread_mutex_unlock(&mutex); // Unlock
    } else {
        // Handle case where mutex is already locked
    }
    return NULL;
}
```

Solution to CS Problem Using Mutex Locks

Mutex with Timeout

```c
#include <pthread.h>
#include <time.h>

pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

void* thread_func(void* arg) {
    struct timespec ts;
    clock_gettime(CLOCK_REALTIME, &ts);
    ts.tv_sec += 1; // Wait for 1 second

    if (pthread_mutex_timedlock(&mutex, &ts) == 0) { // Lock with timeout
        // Critical section
        pthread_mutex_unlock(&mutex); // Unlock
    } else {
        // Handle timeout
    }
    return NULL;
}
```

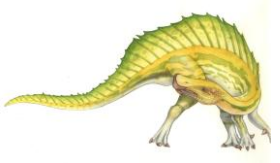
Semaphore

- Synchronization tool that provides more sophisticated ways (than Mutex locks) for processes to synchronize their activities.
- Semaphore ***S*** – integer variable
- Can only be accessed via two indivisible (atomic) operations
  - **`wait()`** and **`signal()`**
    - Originally called **`P()`** and **`V()`**
- Definition of the **`wait()` operation**

```
wait(S) {
    while (S <= 0)
        ; // busy wait
    S--;
}
```

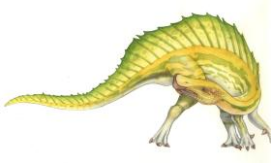
- Definition of the **`signal()` operation**

```
signal(S) {
    S++;
}
```



Semaphore (Cont.)

- **Counting semaphore** – integer value can range over an unrestricted domain
- **Binary semaphore** – integer value can range only between 0 and 1
  - Same as a **mutex lock**
- Can implement a counting semaphore ***S*** as a binary semaphore
- With semaphores we can solve various synchronization problems



Semaphore Usage Example

Semaphore creation in main() common to all examples

```
int main() {
    sem_init(&sem, 0, 1); // Initialize semaphore to 1
    pthread_t t1, t2;
    pthread_create(&t1, NULL, thread_func, NULL);
    pthread_create(&t2, NULL, thread_func, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    sem_destroy(&sem); // Cleanup
    return 0;
}
```

```
int sem_init(sem_t *sem, int pshared, unsigned int value);
```

pshared determines whether the semaphore is shared between threads or processes. 0 for thread synchronization.

Semaphore Usage Example

Basic Mutual Exclusion - Ensure only one thread accesses a critical section at a time.

```
sem_t sem;
int shared_data = 0;

void* thread_func(void* arg) {
    sem_wait(&sem); // Lock
    shared_data++; // Critical section
    sem_post(&sem); // Unlock
    return NULL;
}

int main() {
    sem_init(&sem, 0, 1); // Initialize semaphore to 1
```

Semaphore Usage Example

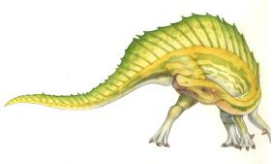
Ordering Thread Execution - Ensure one thread executes before another.

```
void* thread1(void* arg) {
    printf("Thread 1\n");
    sem_post(&sem); // Signal Thread 2
    return NULL;
}

void* thread2(void* arg) {
    sem_wait(&sem); // Wait for Thread 1
    printf("Thread 2\n");
    return NULL;
}

int main() {
    sem_init(&sem, 0, 0); // Initialize semaphore to 0
```

Semaphore Usage Example

```c
void* thread_func(void* arg) {
    sem_wait(&sem); // Acquire semaphore
    printf("Thread %ld accessing resource\n", (long)arg);
    sleep(1); // Simulate work
    sem_post(&sem); // Release semaphore
    return NULL;
}

int main() {
    sem_init(&sem, 0, MAX_THREADS); // Initialize semaphore to MAX_
    pthread_t threads[MAX_THREADS * 2];
    for (long i = 0; i < MAX_THREADS * 2; i++) {
        pthread_create(&threads[i], NULL, thread_func, (void*)i);
    }
    for (int i = 0; i < MAX_THREADS * 2; i++) {
        pthread_join(threads[i], NULL);
    }
    sem_destroy(&sem); // Cleanup
    return 0;
}
```

Limiting Concurrent Access - Limit the number of threads accessing a resource simultaneously.



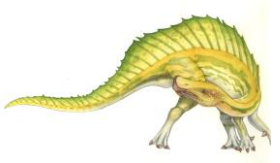
Barrier Synchronization - Wait for all threads to reach a point before proceeding.

```c
#define NUM_THREADS 3

sem_t barrier;

void* thread_func(void* arg) {
    printf("Thread %ld reached barrier\n", (long)arg);
    sem_post(&barrier); // Signal arrival
    sem_wait(&barrier); // Wait for others
    printf("Thread %ld passed barrier\n", (long)arg);
    return NULL;
}

int main() {
    sem_init(&barrier, 0, 0); // Initialize semaphore to 0
    pthread_t threads[NUM_THREADS];
    for (long i = 0; i < NUM_THREADS; i++) {
        pthread_create(&threads[i], NULL, thread_func, (void*)i);
    }
    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }
    sem_destroy(&barrier); // Cleanup
    return 0;
}
```

Ordering two statement in two different processes using a semaphore

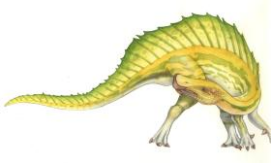
- Consider P_1 and P_2 that with two statements S_1 and S_2 and the requirement that S_1 to happen before S_2
  - Create a semaphore "**synch**" initialized to 0

```
P1:
    S1;
    signal(synch);
P2:
    wait(synch);
    S2;
```



Semaphore Implementation with no Busy waiting

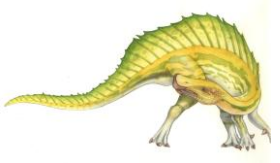
- With each semaphore there is an associated waiting queue
- Each entry in a waiting queue has two data items:
  - Value (of type integer)
  - Pointer to next record in the list
- Two operations:
  - **block** – place the process invoking the operation on the appropriate waiting queue
  - **wakeup** – remove one of processes in the waiting queue and place it in the ready queue



Implementation with no Busy waiting (Cont.)

```
wait(semaphore *S) {
    S->value--;
    if (S->value < 0) {
        add this process to S->list;
        block();
    }
}

signal(semaphore *S) {
    S->value++;
    if (S->value <= 0) {
        remove a process P from S->list;
        wakeup(P);
    }
}
```

```
typedef struct {
  int value;
  struct process *list;
  } semaphore;
```



6.8 Liveness

- Liveness is a property of concurrent systems ensuring that desired events or actions eventually occur, despite delays or interference from other threads. It guarantees progress and prevents scenarios like:
  - **Deadlock**: Threads block indefinitely, each holding a resource needed by another.
  - **Starvation**: A thread is perpetually denied resources, preventing progress.
  - **Livelock**: Threads keep retrying an operation but make no progress due to repeated interference.

6.8.1 Deadlock

The implementation of a semaphore with a waiting queue may result in a situation where two or more processes are waiting indefinitely for an event that can be caused only by one of the waiting processes. The event in question is the execution of a `signal()` operation. When such a state is reached, these processes are said to be **deadlocked**.

To illustrate this, consider a system consisting of two processes, P_0 and P_1, each accessing two semaphores, `S` and `Q`, set to the value 1:

```
        P0                  P1
    wait(S);            wait(Q);
    wait(Q);            wait(S);
       .                   .
       .                   .
       .                   .
    signal(S);          signal(Q);
    signal(Q);          signal(S);
```

Suppose that P_0 executes `wait(S)` and then P_1 executes `wait(Q)`. When P_0 executes `wait(Q)`, it must wait until P_1 executes `signal(Q)`. Similarly, when P_1 executes `wait(S)`, it must wait until P_0 executes `signal(S)`. Since these `signal()` operations cannot be executed, P_0 and P_1 are deadlocked.

Pthread C code for Deadlock

```c
void* thread1(void* arg) {
    sem_wait(&sem1); // Lock sem1
    sem_wait(&sem2); // Wait for sem2 (deadlock)
    printf("Thread 1\n");
    sem_post(&sem2);
    sem_post(&sem1);
    return NULL;
}
```

```c
void* thread2(void* arg) {
    sem_wait(&sem2); // Lock sem2
    sem_wait(&sem1); // Wait for sem1 (deadlock)
    printf("Thread 2\n");
    sem_post(&sem1);
    sem_post(&sem2);
    return NULL;
}
```

- **Thread 1** locks `sem1` and waits for `sem2`.
- **Thread 2** locks `sem2` and waits for `sem1`.
- Both threads block indefinitely, causing a **deadlock**.

End of Chapter 6